

Assembly Language Lab

Registers & Instruction-by-Instruction Debugging in MARS

Duration:	60 minutes
Software:	MARS (MIPS Assembler and Runtime Simulator)
Prerequisites:	Basic understanding of registers, <code>li</code> , <code>add</code> , and <code>sub</code>
Not Covered Yet:	<code>.data</code> , <code>.text</code> , variables, memory, <code>lw</code> , <code>sw</code>

1. Learning Outcomes

By the end of this lab, students will be able to:

1. Trace register values after each instruction.
2. Predict the result of `li`, `add`, and `sub`.
3. Execute a MIPS program one instruction at a time using MARS.
4. Identify and correct simple instruction errors.
5. Explain how the destination register of an instruction is affected.
6. Translate a longer arithmetic expression into a sequence of MIPS instructions.

2. Quick Recap

5 min

Before beginning the activities, review the three instructions used in today's lab.

2.1 `li` — Load Immediate

The `li` instruction places a constant value directly into a register.

```
1 li $t0, 10
```

After execution:

$$\$t0 = 10$$

2.2 `add` — Addition

The `add` instruction adds the values in two source registers and stores the result in a destination register.

```
1 add $t2, $t0, $t1
```

This means:

$$\$t2 = \$t0 + \$t1$$

2.3 sub — Subtraction

The `sub` instruction subtracts the second source register from the first source register.

```
1 sub $t2, $t0, $t1
```

This means:

$$\$t2 = \$t0 - \$t1$$

Remember

For `add` and `sub`, remember:

destination,	source1,	source2
--------------	----------	---------

For example:

```
1 add $t2, $t0, $t1
```

means:

$$\$t2 = \$t0 + \$t1$$

The destination register is where the result is stored.

3. MARS Demonstration

10 min

Run the following program **one instruction at a time** using the **Step / Execute** feature in MARS.

```
1 li $t0, 5
2 li $t1, 8
3 add $t2, $t0, $t1
4 sub $t3, $t2, $t0
```

3.1 Predict Before Stepping

Before running the program, predict what will happen after each instruction.

Step	Instruction	\$t0	\$t1	\$t2	\$t3
0	Start	0	0	0	0
1	li \$t0, 5	5	0	0	0
2	li \$t1, 8	5	8	0	0
3	add \$t2, \$t0, \$t1	5	8	13	0
4	sub \$t3, \$t2, \$t0	5	8	13	8

Check

Compare your predictions with the register values displayed in MARS.

same

Important

Do not simply run the complete program and look at the final answer.
The purpose of this activity is to understand what happens **after each individual instruction**.

Lab Method

Predict → Step → Observe → Explain

4. Guided Activity 1 — Register Tracing

15 min

Before using MARS, predict the register values for the following program.

```

1  li $t0, 15
2  li $t1, 6
3  sub $t2, $t0, $t1
4  add $t3, $t2, $t1

```

4.1 Prediction Table

Step	Instruction	\$t0	\$t1	\$t2	\$t3
0	Start	0	0	0	0
1	li \$t0, 15	15	0	0	0
2	li \$t1, 6	15	6	0	0
3	sub \$t2, \$t0, \$t1	15	6	9	0
4	add \$t3, \$t2, \$t1	15	6	9	3

4.2 Now Use MARS

Execute the program one instruction at a time in MARS.

- Which register changes after each instruction?

1. t0 2. t1 3. t2 4. t3

- Did MARS produce the values you predicted?

yes

- What is the final value of \$t3?

t3 = 3

5. Guided Activity 2 — Find the Bug

15 min

The programmer expects the final value of \$t3 to be 14, but the program contains an error.

```

1  li $t0, 10
2  li $t1, 4
3  add $t2, $t0, $t1

```

```

4  sub $t3, $t0, $t2
5  add $t4, $t1, $t3

```

Your Task

1. Predict the values of registers `$t0` through `$t4`. t0=10 t1=4, t2=14, t3=-4, t4=0
2. Execute the program step by step in MARS. done
3. Identify the instruction that produces the incorrect value for `$t3`. 4th instructions
4. Explain why the instruction is incorrect. its doing 10-14= -4
5. Correct the instruction so that `$t3 = 14`. done

5.1 Corrected Instruction

Write the corrected instruction below:

```
sub $t3, $t2,$t3      # t3= 14-0
```

5.2 Think About It

- Did MARS make a mistake? no
- Did the CPU make a mistake? no
- Or did the program execute exactly what we instructed it to execute? yes, but user made a mistake

Remember

A computer does not understand what the programmer *intended*. It executes the instructions exactly as they are written.

6. Challenge Activity — Changing Register Values

10 min

Trace the following program carefully.

```

1  li $t0, 7
2  li $t1, 3
3  add $t2, $t0, $t1
4  sub $t0, $t2, $t1
5  add $t3, $t0, $t2

```

6.1 Predict First

Do **not** run the program yet.

1. Final value of `$t0`:

7

2. Final value of `$t1`:

7

3. Final value of \$t2:

10

4. Final value of \$t3:

17

5. Which instruction changed the value of \$t0?

sub \$t0, \$t2, \$t1 # t0= 7 just updated the val

Important

A register's value can change during program execution.
Always keep track of which register is being used as the **destination**.

7. Exit Ticket

5 min

Do this **WITHOUT** using MARS first.

```
1 li $t0, 20
2 li $t1, 5
3 sub $t2, $t0, $t1
4 add $t0, $t2, $t1
5 sub $t3, $t0, $t2
```

t2 == 15

t0 == 20

t3 == 5

1. After sub \$t2, \$t0, \$t1:

\$t2 = 15

2. After add \$t0, \$t2, \$t1:

\$t0 = 20

3. Final value of \$t3:

\$t3 = 5

4. Which register was overwritten?

t0

8. Final Task — Translate a Long Expression

You have now practiced loading values into registers and performing addition and subtraction.

For this final task, combine everything you have learned to evaluate a longer arithmetic expression.

8.1 Given Values

Assume:

$$T_1 = 10, \quad T_2 = 5, \quad T_3 = 20, \quad T_4 = 4$$

Evaluate:

$$T_1 + T_3 - (T_2 + T_2 + T_2 + T_2) + (T_4 + T_4 + T_4) - 15$$

Your Task

Write a MIPS program that:

1. Loads the four values into registers.
2. Calculates the expression step by step.
3. Stores the final result in `$t0`.
4. Uses only the instructions covered in this lab:
 - `li`
 - `add`
 - `sub`

Important

Do not use multiplication or division for this task.

You must construct the expression using only the `add` and `sub` instructions introduced in this lab.

8.2 Step 1 — Plan Your Registers

Decide which registers will hold T_1 , T_2 , T_3 , and T_4 .

Value	Register
$T_1 = 10$	<code>t1</code>
$T_2 = 5$	<code>t2</code>
$T_3 = 20$	<code>t3</code>
$T_4 = 4$	<code>t4</code>

8.3 Step 2 — Break the Expression Down

Calculate each part before writing the complete program.

$$\begin{aligned}
 T_1 + T_3 &= \underline{\text{add } \$t5, \$t1, \$t3} && 30 \\
 T_2 + T_2 + T_2 + T_2 &= \underline{\begin{array}{l} \text{add } \$t2, \$t2, \$t2 \\ \text{add } \$t2, \$t2, \$t2 \end{array}} && 20 \\
 T_4 + T_4 + T_4 &= \underline{\begin{array}{l} \text{add } \$t4, \$t0, \$t4 \\ \text{add } \$t4, \$t4, \$t4 \end{array}} && 12
 \end{aligned}$$

Final Result = 7

8.4 Step 3 — Write Your MIPS Program

```
1 # Write your MIPS program here
```

8.5 Step 4 — Verify Your Answer

Calculate manually:

$$10 + 20 - (5 + 5 + 5 + 5) + (4 + 4 + 4) - 15$$

$$= \underline{7}$$

Now run your program in MARS and check the value stored in \$t0.

\$t0 = 0

8.6 Bonus Challenge

Can you rewrite your program using **fewer instructions** while still following the rules of this lab?

```
1 # Improved version
```

Key Rule

**DON'T RUN FIRST
— PREDICT FIRST!**

Read Instruction → Predict → Step → Observe → Explain

*The goal of this lab is not just to get the correct answer.
The goal is to understand how each instruction changes the machine state.*